

离散数学

刘晓芳

liuxiaofang@nankai.edu.cn



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南开大学

Nankai University

第4章 归纳与递归



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南开大学

Nankai University

主要内容

- 第4章 归纳与递归
 - 4.1 数学归纳法
 - 4.2 强归纳和良序
 - 4.3 递归定义和结构归纳法
 - 4.4 递归算法

4.1 数学归纳法

- 例如：有一个无穷级的梯子，我们想知道我们能否登上每一级梯子。我们知道两件事：
- 1. 我们能登上第一级梯子。
- 2. 如果我们能登上某一级梯子，我们就能登上上一级梯子。
- 由（1），我们能登上第一级梯子，再由（2），我们能登上第二级梯子，再由（2），我们能登上第3级梯子，继续这个论证，我们能登上第4级、第5级梯子。重复100次步骤（2），我们可以登上第101级梯子。反复地应用步骤（2），我们可以登上第 n 级梯子，对任意正整数 n 成立。

4.1 数学归纳法

- 4.1.1 数学归纳法
- 4.1.2 用数学归纳法证明的例子
- 4.1.3 为什么数学归纳法合理?
- 4.1.4 归纳法证明中常见的错误

4.1.1 数学归纳法

- 数学归纳法的步骤：要证明 $P(n)$ 对所有正整数 n 成立，其中 $P(n)$ 是一个命题函数，我们需要两个步骤：
- 归纳基础：我们验证 $P(1)$ 为真。
- 归纳步：我们证明蕴涵式 $P(k) \rightarrow P(k+1)$ 对所有正整数 k 成立。
- *假设 $P(k)$ 为真,称为归纳假设。
- *用谓词公式表示数学归纳法：

$$[P(1) \wedge \forall k(P(k) \rightarrow P(k+1))] \rightarrow \forall nP(n)$$

4.1.1 数学归纳法

- 理解数学归纳法
- (1)登无穷级的梯子
- (2) n 个人传秘密
- (3)多米诺骨牌

4.1.2 用数学归纳法证明的例子

➤ 1. 证明和式

➤ 例1：证明前 n 个正整数的和： $1 + 2 + \cdots + n = \frac{n(n+1)}{2}$

➤ 解：设 $P(n)$ 表示前 n 个正整数的和为 $\frac{n(n+1)}{2}$ 。

➤ 归纳基础：验证 $P(1)$ 为真， $1 = \frac{1 \cdot (1+1)}{2}$ 。

➤ 归纳步：作归纳假设，设 $P(k)$ 为真，即

➤ $1 + 2 + \cdots + k = \frac{k(k+1)}{2}$ ，在这个假设下证明 $P(k+1)$ 为真。

➤ $1 + 2 + \cdots + k + (k + 1) = \frac{k(k+1)}{2} + (k + 1) = \frac{k(k+1) + 2(k+1)}{2} = \frac{(k+1)(k+2)}{2}$ ，故 $P(k+1)$ 为真。

故 $P(n)$ 对任意正整数 n 成立。

4.1.2 用数学归纳法证明的例子

- 例2：给出前 n 个奇正整数的和的猜想，并用数学归纳法证明。
- 解： $1 = 1, 1 + 3 = 4, 1 + 3 + 5 = 9, 1 + 3 + 5 + 7 = 16,$
- $1 + 3 + 5 + 7 + 9 = 25$, 因此我们猜想：
- $1 + 3 + \cdots + (2n - 1) = n^2$, 用归纳法证明：
- 归纳基础：验证 $P(1)$ 成立。 $1 = 1^2$, 结论成立。
- 归纳步：假设 $1 + 3 + \cdots + (2k - 1) = k^2$, 即 $P(k)$ 为真，证明 $P(k+1)$ 为真。
- $1 + 3 + \cdots + (2k - 1) + (2k + 1) = k^2 + 2k + 1 = (k + 1)^2$
- 故 $P(k) \rightarrow P(k + 1)$ 成立。所以 $P(n)$ 对任意正整数 n 成立。

4.1.2 用数学归纳法证明的例子

➤ 例4：（几何级数的和式）

➤ $\sum_{j=0}^n ar^j = a + ar + ar^2 + \cdots + ar^n = \frac{ar^{n+1} - a}{r-1}, (r \neq 1)$

➤ 解：归纳基础：n=0时， $a = \frac{ar^{0+1} - a}{r-1} = \frac{a(r-1)}{r-1}$

➤ 归纳步：假设n=k时，公式成立，即

$$a + ar + \cdots + ar^k = \frac{ar^{k+1} - a}{r-1}$$

➤ 证明n=k+1时，公式仍成立。

➤
$$a + ar + \cdots + ar^k + ar^{k+1} = \frac{ar^{k+1} - a}{r-1} + ar^{k+1} = \frac{ar^{k+1} - a}{r-1} + \frac{ar^{k+2} - ar^{k+1}}{r-1} = \frac{ar^{k+2} - a}{r-1}$$

故公式对所有非负整数成立。

4.1.2 用数学归纳法证明的例子

➤ 2. 证明不等式

➤ 例5：证明： $n < 2^n$ 对所有正整数 n 成立。

➤ 解：归纳基础：当 $n=1$ 时， $1 < 2^1$ ，结论成立。

➤ 归纳步：假设 $n=k$ 时，有 $k < 2^k$ 成立。

➤ 当 $n=k+1$ 时， $k+1 < 2^k + 1 < 2^k + 2^k = 2^{k+1}$ ，故结论成立。

4.1.2 用数学归纳法证明的例子

- 例6：用数学归纳法证明： $2^n < n!$ 对所有 $n \geq 4$ 的正整数 n 成立。（注意： $n=1,2,3$ 时，该不等式不成立）
- 解：设 $P(n)$ 表示 $2^n < n!$ 。
- 归纳基础： $n=4$ 时， $2^4 = 16 < 4! = 24$ ，故 $P(4)$ 成立。
- 归纳步：假设 $P(k)$ 为真，即 $2^k < k!$ ($k \geq 4$)。
- 证明 $P(k+1)$ 也为真。 $2^{k+1} = 2 \cdot 2^k < 2 \cdot k! < (k+1)k! = (k+1)!$ 故 $P(n)$ 对所有 $n \geq 4$ 的正整数 n 成立。

4.1.2 用数学归纳法证明的例子

- 例7：调和数的不等式。
- 调和数 $H_j = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{j}$
- 证明： $H_{2^n} \geq 1 + \frac{n}{2}$

4.1.2 用数学归纳法证明的例子

- 解：归纳基础： $n=0$ 时， $H_{2^0} = H_1 = 1 \geq 1 + \frac{0}{2} = 1$.
- 归纳步：假设 $n=k$ 时，结论成立。即 $H_{2^k} \geq 1 + \frac{k}{2}$.
- 欲证： $H_{2^{k+1}} \geq 1 + \frac{k+1}{2}$ 。
- $H_{2^{k+1}} = 1 + \frac{1}{2} + \cdots + \frac{1}{2^k} + \frac{1}{2^{k+1}} + \cdots + \frac{1}{2^{k+1}}$
- $= H_{2^k} + \frac{1}{2^{k+1}} + \cdots + \frac{1}{2^{k+1}}$
- $\geq \left(1 + \frac{k}{2}\right) + \frac{1}{2^{k+1}} + \cdots + \frac{1}{2^{k+1}}$
- $\geq \left(1 + \frac{k}{2}\right) + 2^k \cdot \frac{1}{2^{k+1}} \geq \left(1 + \frac{k}{2}\right) + \frac{1}{2} = 1 + \frac{k+1}{2}$ 。

故 $H_{2^n} \geq 1 + \frac{n}{2}$ 对所有非负整数 n 成立. (注意 H_j 是一个发散的级数)。

4.1.2 用数学归纳法证明的例子

➤ 3. 证明可整除性

- 例8：证明： $n^3 - n$ 可被3整除，其中 n 为正整数。
- 证明：归纳基础： $n=1$ 时， $1^3 - 1 = 0$ 可被3整除。
- 归纳步：假设 $n=k$ 时， $k^3 - k$ 可被3整除($k \geq 1$)。
- 当 $n=k+1$ 时， $(k+1)^3 - (k+1) = (k^3 + 3k^2 + 3k + 1) - (k+1) = (k^3 - k) + 3(k^2 + k)$ 。已知 $k^3 - k$ 能被3整除，又因为 $3(k^2 + k)$ 也能被3整除，故 $(k+1)^3 - (k+1)$ 能被3整除。

4.1.2 用数学归纳法证明的例子

- 4. 证明集合的有关结果
- 例9：（有穷集合的子集数）设集合S有n个元素（n为非负整数），那么S有 2^n 个子集。

4.1.2 用数学归纳法证明的例子

- 解：设 $P(n)$: n 个元素的集合 S 有 2^n 个子集。
- 归纳基础： $n=0$ 时， S 只有一个子集：空集，故 S 有 $2^0 = 1$ 个子集。
- 归纳步：假设 $n=k$ 时， $P(k)$ 为真，即有 k 个元素的集合 S 有 2^k 个子集。
- 当 $n=k+1$ 时，设 T 有 $k+1$ 个元素， a 是 T 中某一个元素，有 $T=S \cup \{a\}$ 或 $S= T - \{a\}$ ，那么 S 中有 k 个元素。由归纳假设， S 有 2^k 个子集，对于 S 中的每一个子集 X ， T 中有两个不同的子集 X 和 $X \cup \{a\}$ 。而 S 有 2^k 个子集，故 T 中有 $2 \cdot 2^k = 2^{k+1}$ 个不同的子集

4.1.2 用数学归纳法证明的例子

- 5. 创造性地使用归纳法
- 例12：奇数个人扔饼游戏。
- 有奇数个人站在院子里，任意两个人的距离互不相同。所有人同时朝距离他最近的人扔饼。证明：总有一个不会被饼打到。

4.1.2 用数学归纳法证明的例子

- 解：设： $P(n)$:表示 $2n+1$ 个人中总有一个人不会被饼打到。
- 归纳基础：当 $n=1$ 时，共有 $2n+1=3$ 个人。设有A,B,C三个人，假设A和B距离最近，那么A和B互相扔饼，C朝A或B扔饼，因此，C不会被饼打到。
- 归纳步：假设 $n=k$ 时， $P(k)$ 成立。
- 现在设 $n=k+1$ ，共有 $2(k+1)+1=2k+3$ 个人。设其中A和B两个人距离最近，故A和B两人互相扔饼，剩下的 $2k+1$ 人中如果有一个朝A或B扔饼，共有 $2k+1$ 人，而扔向这 $2k+1$ 人的饼至多有 $2k$ 个，故有一人不会被饼打到。若这 $2k+1$ 人中没有人向A和B扔饼，由归纳假设， $2k+1$ 人中总有一人不会被饼打到。在任何情况下，这 $2k+3$ 个人中总有一人不会被饼打到。故 $P(k+1)$ 成立。



4.1.2 用数学归纳法证明的例子

- 例13：设 n 是正整数。证明 $2^n \times 2^n$ 棋盘删去其中一个方格后总能被直角三米诺填充。
- 课后练习

4.1.3 为什么数学归纳法合理？

- 1. 良序性质：每一个元素是正整数的集合有一个最小元。
- 2. 数学归纳法的合理性证明：数学归纳法证明 $P(1)$ 成立，并对任意正整数 k ，由 $P(k)$ 为真推出 $P(k+1)$ 为真。假若 $P(n)$ 不是对所有正整数 n 成立。令 $S=\{m|m\text{是正整数且}P(m)\text{为假}\}$ 。显然 $1 \notin S$ ，因为已证 $P(1)$ 成立。由良序性质， S 中存在最小元 $k+1$ ，因而 $k \notin S$ 且 $P(k)$ 成立。由归纳法知， $P(k)$ 为真蕴含 $P(k+1)$ 为真，因而 $k+1 \notin S$ ，这与 $k+1$ 是 S 中的最小元矛盾。

4.1.4 归纳法证明中常见的错误

- 例14：证明： n 条两两不平行的直线交于一个公共点。
- “证明：” 设 $P(n)$ 表示 n 条两两不平行的直线交于一个公共点($n \geq 2$)。
- 归纳基础：当 $n=2$ 时，由平行线的定义，2条不平行的直线交于一点。故 $P(2)$ 得证。
- 归纳步：假设 $P(k)$ 为真，现在设 $n=k+1$ 。设 $p_1, p_2, \dots, p_{k+1}$ 是两两不平行的 $k+1$ 条直线。考虑 $p_1, p_2, \dots, p_k$ ，由归纳假设，它们交于一个公共点 M ，再考虑 $p_2, p_3, \dots, p_{k+1}$ ，由归纳假设，它们交于一个公共点 N 。若 M 和 N 不同，则经过 M 和 N 的直线只能有一条，但 $p_2, p_3, \dots, p_k$ 都经过 M 和 N ，故 M 和 N 是同一点。证毕。

4.1.4 归纳法证明中常见的错误

- 解：这个证明不适用于 $k=2$ 的情形。当 $k=2$, $p_2, p_3, \dots, p_k$ 只有一条直线 p_2 , 上述证明 M 等于 N 不成立。

4.2 强归纳和良序

- 4.2.1 强归纳法 (strong induction)
- 4.2.2 用强归纳法证明的例子
- 4.2.3 使用良序性质证明

4.2.1 强归纳法

- **强归纳法**：要证明 $P(n)$ 对所有正整数 n 成立，其中 $P(n)$ 是一个命题函数，我们需要以下两步：
- **归纳基础**：我们验证 $P(1)$ 为真。
- **归纳步**：我们证明蕴涵式 $[P(1) \wedge P(2) \wedge \cdots \wedge P(k)] \rightarrow P(k+1)$ 对所有正整数 k 成立。
- ***强归纳法与数学归纳法是等价的。**
- ***任何用数学归纳法能证明的定理，用强归纳法都能证，因为数学归纳法的假设 $P(k)$ 是强归纳法的假设 $P(1) \wedge P(2) \wedge \cdots \wedge P(k)$ 中的一部分，如果 $P(k) \rightarrow P(k+1)$ ，则必有 $P(1) \wedge P(2) \wedge \cdots \wedge P(k) \rightarrow P(k+1)$ 。**

4.2.1 强归纳法

- *强归纳法有时称为第二归纳原理(second principle of mathematical induction)或完全归纳法(complete induction)。
- *强归纳法和无穷梯子
- 强归纳法告诉我们，如果我们做到以下两步，我们可以登上无穷梯子的每一级。
- 我们能登上第一级梯子；
- 对每一个正整数 k ，如果我们能登上梯子的前 k 级，那么我们能登上梯子的第 $k+1$ 级。

4.2.1 强归纳法

- 例1：如果我们能登上无穷梯子的第一、第二级，并且我们知道，如果我们能登上梯子的第 k 级，那么我们能登上梯子的第 $k+2$ 级。用强归纳法证明，我们能登上梯子的每一级。

4.2.1 强归纳法

- 解：*用数学归纳法没办法证明。
- 用强归纳法证明：
- 归纳基础：首先我们验证我们能登上梯子的第一、第二级。
- 归纳步：假设我们可以登上梯子的第1级，第2级， $\dots$ ，第 k 级 ($k \geq 2$)，现在考虑第 $k+1$ 级。因为我们可以登上第 $k-1$ 级（由归纳假设），由我们题目的条件知，我们可以登上第 $(k-1) + 2 = k + 1$ 级。故结论得证。



4.2.2 用强归纳法证明的例子

- 例2：证明：如果 n 是一个大于1的整数，那么 n 可以写成素数的乘积.

4.2.2 用强归纳法证明的例子

- 解：设 $P(n)$ 表示 n 可以写成素数的乘积。
- 归纳基础： $P(2)$ 成立，因为2可以写成一个素数，即它自己。
- 归纳步：假设对所有 $1 < j \leq k$ 的正整数 j 有 $P(j)$ 成立。当 $n=k+1$ 时，如果 $k+1$ 是一个素数，那么 $k+1$ 可以写成它自己。若 $k+1$ 是一个合数，那么 $k+1=a \cdot b$ ，其中 $1 < a \leq k$ 且 $1 < b \leq k$ ，由归纳假设，知 a 可以写成素数的乘积， b 也可以写成素数的乘积，故 $k+1=a \cdot b$ 也可以写成 a 中的那些素数和 b 中的那些素数的乘积。

4.2.2 用强归纳法证明的例子

- 例3：（取火柴游戏）假设有两堆一样多的火柴，由两个人轮流取火柴，每人每次从一堆中取任意正整数根火柴，最后取火柴的那个人赢。证明：第二个取的人总能够赢。

4.2.2 用强归纳法证明的例子

- 解：设一开始两堆火柴中各有 n 根火柴。设 $P(n)$ 表示在这种情况下，第二个人一定能赢。
- 归纳基础：当 $n=1$ 时，第一个人在一堆中取一根火柴，第二个人只需在另一堆中取一根火柴，就赢了。故 $P(1)$ 成立。
- 归纳步：假设两堆火柴各有 j 根($1 \leq j \leq k$)时， $P(j)$ 成立。
- 现在设两堆火柴各有 $k+1$ 根。若第一个人在一堆中取 $k+1$ 根火柴，则第二个人在剩下的另一堆中也取 $k+1$ 根，则第二个人就赢了。如果第一个人在一堆中取 r ($1 \leq r < k+1$)根火柴，那么第二个人也在另一堆火柴中取 r 根，这时，两堆火柴各剩 $k+1-r < k+1$ 根，由归纳假设， $P(k+1-r)$ 成立，故第二个人总能赢。

4.2.2 用强归纳法证明的例子

- 例4：证明总数大于或等于12分的邮费可仅用4分和5分的邮票表示。
- 解：设 $P(n)$ 表示 n 分钱的邮费可用4分和5分的邮票表示。
- 用数学归纳法证明：

4.2.2 用强归纳法证明的例子

- 归纳基础：12分的邮费可用3张4分的邮票表示。
- 归纳步：假设 $P(k)$ 为真。现在证 $n=k+1$ 时， $P(n)$ 为真。因为 k 分钱的邮票可用4分和5分的邮票表示，若其中至少有一个4分的邮票，那么将它换成5分的邮票就证明了 $P(k+1)$ 。若 k 分钱邮费全部是5分钱的邮票，因为 $k \geq 12$ ，全部是5分的邮票则 $k \geq 15$ ，其中至少有3张5分的邮票，将其中3张5分的邮票换成4张4分的邮票，就可以表示 $k+1$ 分钱。故 $P(k+1)$ 成立。

4.2.2 用强归纳法证明的例子

- 用强归纳法证明：
- 归纳基础：可以验证12，13，14和15分钱可用3张4分，2张4分1张5分，1张4分2张5分和3张5分邮票表示。故 $P(12)$ ， $P(13)$ ， $P(14)$ 和 $P(15)$ 成立。
- 归纳步：由归纳假设知 $P(j)$ 对 $12 \leq j \leq k$ 成立且 $k \geq 15$ 。要证 $P(k+1)$ 成立。因为 $P(k-3)$ 成立且 $k-3 \geq 12$ ，因此，在 $k-3$ 分钱的邮票中加一张4分的邮票，就能表示 $k+1$ 分钱的邮费了，故 $P(k+1)$ 成立。

4.2.3 使用良序性质证明

- 1. 良序性质(Well-ordering property): 每个非空的非负整数的集合都一定有一个最小元。
- *良序性质和数学归纳法、强归纳法都是等价的, 其中任意一个的正确性都可以用另外两个之一来证明。

4.2.3 使用良序性质证明

- 例5：用良序性质证明除算法。即：如果 a 是一个整数且 d 是一个正整数，那么存在唯一的整数 q 和 r ，使得 $a=dq+r$ 且 $0 \leq r < d$ 。
- 解：设 S 是具有形式 $a - dq$ 的所有非负整数的集合，其中 q 是一个整数。这个集合 S 是非空集，因为我们可以取 q 为负整数，因而使 $-dq$ 尽可能大。由良序性质， S 有一个最小元 $r = a - dq_0$ 。

4.2.3 使用良序性质证明

- 由集合 S 的要求, r 为非负数, 同时有 $r < d$ 。假若不然, 有 $r \geq d$ 。由于 $a = dq_0 + r$, 因而有 $a - d(q_0 + 1) = (a - dq_0) - d = r - d \geq 0$, $r - d$ 是 S 中比 r 更小的元素, 这与 r 是 S 中的最小元的结论矛盾。故存在 $q = q_0$ 和 $r(0 \leq r < d)$ 使得 $a = dq + r$ 。
- 再证 r 和 q 是唯一的。由于 r 是满足 $a - dq$ 的最小非负整数, 假若还有一个 r_1 也满足 $a - dq_1$ 是非负整数且 $0 \leq r_1 < d$, 若 $r_1 < r$, 这与 r 是最小元的假设矛盾; 若 $r_1 > r$, 由于 $r_1 = a - dq_1 > r = a - dq$, 从而有 $q > q_1$, 即 $q \geq q_1 + 1$, 从而 $r_1 \geq r + d$, 这与 $0 \leq r_1 < d$ 矛盾。故 $r_1 = r$ 。由 $r_1 = a - dq_1 = r = a - dq$ 知, $q_1 = q$ 。从而 r 和 q 是唯一的。

作业13

- 1. 用数学归纳法证明和式: $1^3 + 2^3 + \cdots + n^3 = \left[\frac{n(n+1)}{2}\right]^2$.
- 2. 用数学归纳法证明: 2整除 $n^2 + n$, 其中 n 是一个正整数。
- 3. 设 $P(n)$ 表示 n 分钱的邮费可以只用4分和7分的邮票表示。用强归纳法证明 $P(n)$ 对 $n \geq 18$ 的所有整数成立。
- 4. 用强归纳法证明每一个正整数 n 可以写成2的不同的幂的和。

4.3 递归定义和结构归纳法

- 4.3.1 递归定义的函数
 - 4.3.2 递归定义的集合与结构
 - 4.3.3 结构归纳法
-
- *有时很难给出一个对象的显式定义，但很容易用对象自己来定义该对象，这种方法称为递归 (recursion)

4.3.1 递归定义的函数

- 我们用两步定义一个以非负整数为定义域的函数：
- 基础步：说明函数在0处的值。
- 递归步：根据函数在较小的整数的值定义函数在更大的整数的值。
- 这种定义称为递归的(recursive)或归纳定义(inductive definition)。

4.3.1 递归定义的函数

- 例1：给出以下递归定义的函数
- $f(0)=3$
- $f(n+1)=2f(n)+3$
- 求： $f(1)$, $f(2)$, $f(3)$, $f(4)$ 。
- 解： $f(1) = 2f(0) + 3 = 2 \cdot 3 + 3 = 9$
- $f(2) = 2f(1) + 3 = 2 \cdot 9 + 3 = 21$
- $f(3) = 2f(2) + 3 = 2 \cdot 21 + 3 = 45$
- $f(4) = 2f(3) + 3 = 2 \cdot 45 + 3 = 93$

4.3.1 递归定义的函数

- 例2：给出阶乘函数 $F(n)=n!$ 的归纳定义。
- 解： $F(0)=0!=1$.
- $F(n+1)=(n+1)F(n)$
- 例如： $F(5)=5F(4)=5 \cdot 4F(3)=5 \cdot 4 \cdot 3F(2)=5 \cdot 4 \cdot 3 \cdot 2F(1)$
- $=5 \cdot 4 \cdot 3 \cdot 2 \cdot 1F(0)=5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 \cdot 1=120$ 。
- *一个递归定义的函数称为是良好定义的(well defined), 是说：对每一个正整数 n ，函数在 n 处的值以无二义的方式确定。

4.3.1 递归定义的函数

➤ 例4：递归定义以下和式：

➤ $S_n = \sum_{i=0}^n a_i$

➤ 解： $S_0 = \sum_{i=0}^0 a_i = a_0$

➤ $S_{n+1} = \sum_{i=0}^{n+1} a_i = \sum_{i=0}^n a_i + a_{n+1} = S_n + a_{n+1}$

➤ 定义1：Fibonacci序列 $f_0, f_1, f_2, \dots$ 递归定义为：
 $f_0 = 0, f_1 = 1$, 且 $f_n = f_{n-1} + f_{n-2}, n = 2, 3, 4, \dots$ 。

4.3.1 递归定义的函数

➤ 例5：求Fibonacci数 f_2, f_3, f_4, f_5 和 f_6 。

➤ 解： $f_2 = f_1 + f_0 = 1 + 0 = 1$

➤ $f_3 = f_2 + f_1 = 1 + 1 = 2$

➤ $f_4 = f_3 + f_2 = 2 + 1 = 3$

➤ $f_5 = f_4 + f_3 = 3 + 2 = 5$

➤ $f_6 = f_5 + f_4 = 5 + 3 = 8$ 。

4.3.1 递归定义的函数

- 例6：证明：当 $n \geq 3$ ，有 $f_n > \alpha^{n-2}$ ，其中 $\alpha = (1 + \sqrt{5})/2$ 。
- 解：我们用强归纳法证明这个不等式。设 $P(n)$ 表示 $f_n > \alpha^{n-2}$ 。
- 归纳基础： $n=3, 4$ 时，
- $\alpha < 2 = f_3$, $\alpha^2 = \frac{3+\sqrt{5}}{2} < 3 = f_4$
- 故 $P(3), P(4)$ 成立。

4.3.1 递归定义的函数

- 归纳步：假设 $P(j)$ 成立，即 $f_j > \alpha^{j-2}$ 对所有 $j(3 \leq j \leq k \text{ 且 } k \geq 4)$ 成立。欲证 $P(k+1)$ 成立，即 $f_{k+1} > \alpha^{k-1}$ 。因为 α 是方程 $x^2 - x - 1 = 0$ 的一个解，故有 $\alpha^2 = \alpha + 1$ 。故
- $\alpha^{k-1} = \alpha^2 \cdot \alpha^{k-3} = (\alpha + 1)\alpha^{k-3} = \alpha \cdot \alpha^{k-3} + 1 \cdot \alpha^{k-3}$
 $\alpha^{k-3} = \alpha^{k-2} + \alpha^{k-3}$
- 由归纳假设，如果 $k \geq 4$ ，有
$$f_{k-1} > \alpha^{k-3}, \quad f_k > \alpha^{k-2}$$
- 从而有 $f_{k+1} = f_k + f_{k-1} > \alpha^{k-2} + \alpha^{k-3} = \alpha^{k-1}$ 。

4.3.1 递归定义的函数

- **定理1: (LAME定理)** 设 a, b 是正整数且 $a \geq b$ 。那么用欧几里德算法求 $\gcd(a, b)$ 所用的除法的次数小于或等于5乘以 b 的十进制数的位数。

4.3.2 递归定义的集合与结构

- *递归定义中的基础步与递归步，排斥规则
- 基础步：初始元素
- 递归步：从已知属于集合的元素构造集合的新元素的规则
- 排斥规则：集合仅包含基础步所规定的以及递归步应用所生成的元素（默认成立）

4.3.2 递归定义的集合与结构

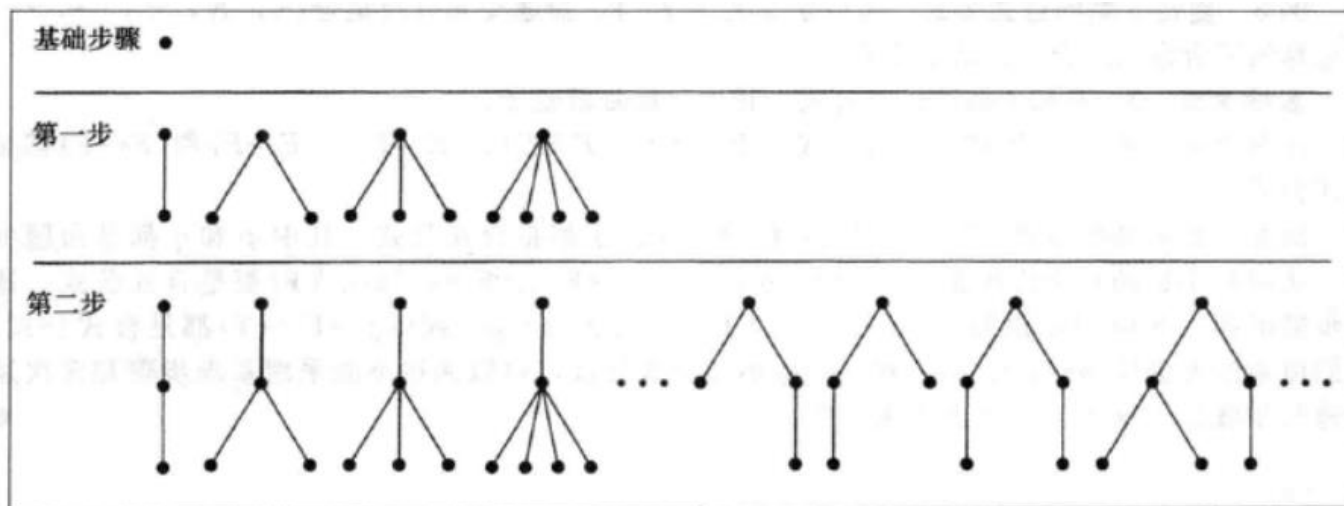
- 例7：考虑整数集合的子集 S ，递归定义如下：
- 基础步： $3 \in S$
- 递归步：如果 $x \in S$ 且 $y \in S$ ，那么 $x + y \in S$ 。
- 那么 $3 + 3 = 6 \in S, 3 + 6 = 9 \in S, 6 + 6 = 12 \in S$, 等等。

4.3.2 递归定义的集合与结构

- 例10：复合命题的合式公式：我们可以用T、F、命题变量和联结词： $\neg$, $\wedge$, $\vee$, $\rightarrow$, $\leftrightarrow$ 定义复合命题的合式公式：
- 基础步：T, F和s都是合式公式，其中s是一个命题变量。
- 递归步：如果E和F是合式公式，那么 $(\neg E)$, $(E \wedge F)$, $(E \vee F)$, $(E \rightarrow F)$, $(E \leftrightarrow F)$ 都是合式公式。
- *排斥规则。
- 由定义知： $(p \vee q)$, $(p \rightarrow F)$, $(q \wedge F)$, $((p \vee q) \rightarrow (q \wedge F))$, $(q \vee (p \wedge q))$, $((p \rightarrow F) \rightarrow T)$ 都是合式公式，而 $p \neg \wedge q$, $pq \wedge$, $\neg \wedge pq$ 都不是合式公式。

4.3.2 递归定义的集合与结构

- **定义4：有根树 (rooted tree)**
- **基础步：** 一个单独的顶点 r 是一个有根树， r 是该树的树根。
- **递归步：** 假设 $T_1, T_2, \dots, T_n$ 是互不相交的有根树，它们的树根分别为 $r_1, r_2, \dots, r_n$ 。由一个新顶点 r 和 $T_1, T_2, \dots, T_n$ 构成一个新的有根树 T ，其中 r 与 $r_1, r_2, \dots, r_n$ 分别连一条边， r 称为 T 的树根。

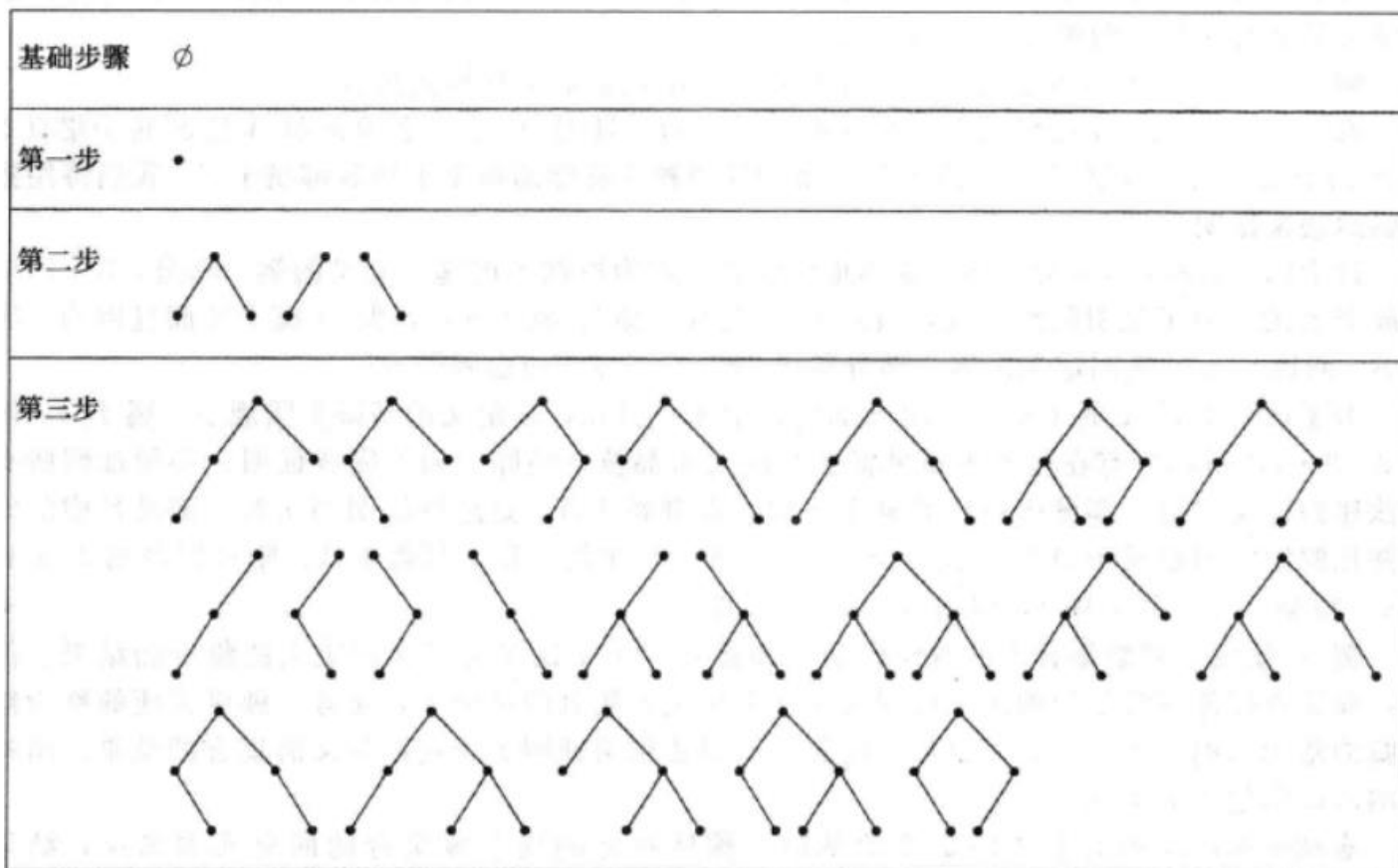


4.3.2 递归定义的集合与结构

- 定义5：一个扩展二叉树(extended binary tree)定义如下：
- 基础步：一个空集是一个扩展二叉树。
- 递归步：如果 T_1 和 T_2 是两个不相交的扩展二叉树，由新顶点 r 和 T_1, T_2 构成一个新的扩展二叉树 $T = T_1 \cdot T_2$ ， r 与 T_1, T_2 的树根各连一条边(当 T_1, T_2 不为空时)。 r 称为 T 的根， T_1 称为 r 的左子树(left subtree)， T_2 称为 r 的右子树(right subtree)。

4.3.2 递归定义的集合与结构

- 通过递归步骤一次至三次建立扩展二叉树

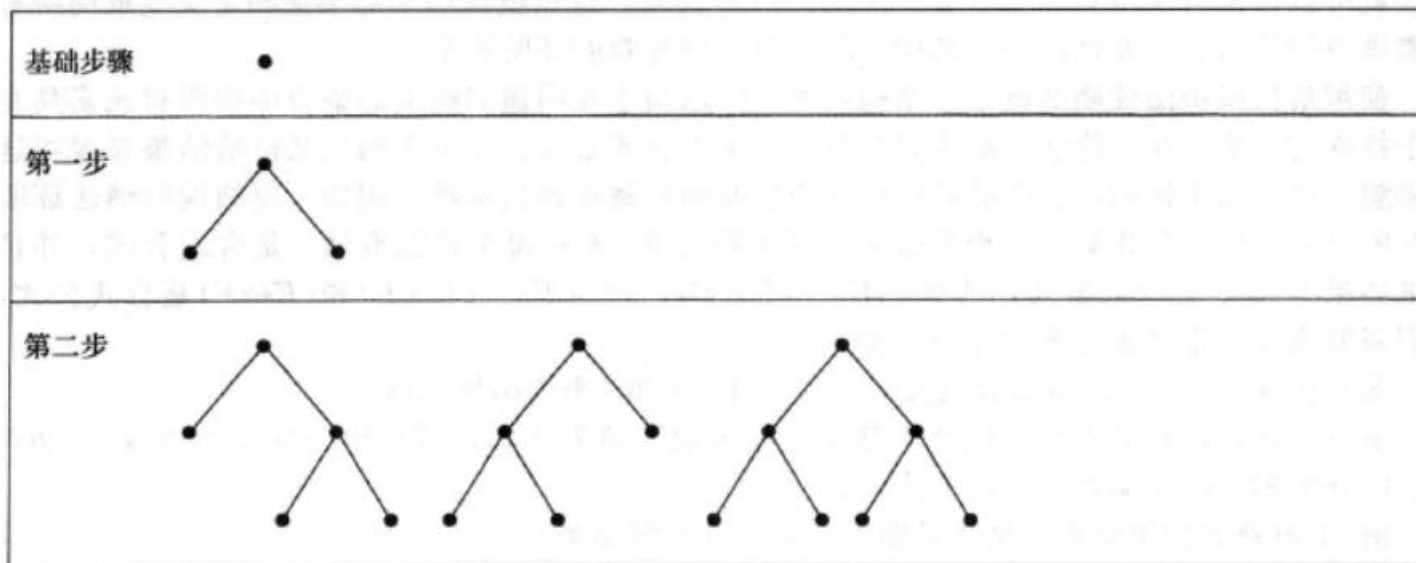


4.3.2 递归定义的集合与结构

- 定义6：满二叉树(full binary tree)
- 基础步：一个单独的顶点 r 是一个满二叉树， r 是该树的根。
- 归纳步：如果 T_1 和 T_2 是不相交的满二叉树，由新顶点 r 和 T_1, T_2 构成一个新的满二叉树 $T = T_1 \cdot T_2$ ，其中 r 与左子树 T_1 的树根及右子树 T_2 的树根各连一条边。
- *满二叉树的每个顶点或者有两个孩子，或者没有孩子。

4.3.2 递归定义的集合与结构

➤ 建立满二叉树



4.3.3 结构归纳法

- 例12：证明例7中的集合S的元素是3的所有正整数倍的数。
- 解：设 $A = \{3n | n \in \mathbb{Z}^+\}$ 。证明： $A=S$ 。
- 首先证： $A \subseteq S$ 。设 $P(n)$ 表示 $3n \in S$ 。
- 归纳基础： $n=1$ 时，由S的定义， $3 = 3 \cdot 1 \in S$ 。
- 归纳步：假设对正整数 k ，有 $P(k)$ 成立，即 $3k \in S$ ，欲证 $P(k+1)$ 成立。由于 $3 \in S$ ，又由假设有 $3k \in S$ ，由S的定义， $3k + 3 = 3(k+1) \in S$ 。故 $P(k+1)$ 成立。
- 再证： $S \subseteq A$ 。由S的定义，首先有 $3 = 3 \cdot 1 \in S$ ，故 $3 \in A$ 。再由归纳步，若 $x, y \in S$ ，有 $x + y \in S$ ，由于 $x \in S$ 和 $y \in S$ ，由归纳假设， $x \in A$ 且 $y \in A$ ，即 $3|x$ 且 $3|y$ ，由3.4节定理1的(i)，有 $3|(x+y)$ ，故 $x+y \in A$ 。

4.3.3 结构归纳法

- *结构归纳法(structural induction)
- 归纳基础：证明递归定义的基础步中属于集合的所有元素结论都成立。
- 递归步：证明：如果在定义的递归步中用于构造新元素的元素都满足结论，那么在定义的递归步中所构造出来的新元素也满足结论。

4.3.3 结构归纳法

- 例13：证明例10中定义的合式公式左、右括号的数目相等。
- 解：基础步：T, F和s不含括号，故左、右括号数目相等。
- 递归步：假设p和q是合式公式，其中左、右括号数目相等，令 l_p 和 l_q 分别表示p和q的左括号数， r_p 和 r_q 分别表示p和q的右括号数。已知 $l_p = r_p$ ， $l_q = r_q$ 。那么在 $(\neg p)$ 中，左括号数 $= l_p + 1 = r_p + 1 =$ 右括号数，而在 $(p \vee q)$ ， $(p \wedge q)$ ， $(p \rightarrow q)$ 和 $(p \leftrightarrow q)$ 中，左括号数 $= l_p + l_q + 1 = r_p + r_q + 1 =$ 右括号数，故由结构归纳法知，所有合式公式的左、右括号数目相等。

4.3.3 结构归纳法

- 定义7: 定义满二叉树 T 的高度 $h(T)$ 如下:
- 基础步: 只有一个根结点 r 的满二叉树 T 的高度 $h(T)=0$ 。
- 递归步: 如果 T_1, T_2 是满二叉树, $T = T_1 \cdot T_2$, 那么 T 的高度 $h(T) = 1 + \max(h(T_1), h(T_2))$ 。
- 设 $n(T)$ 表示满二叉树 T 的顶点数, 递归定义如下:
- 基础步: 只含一个根结点 r 的满二叉树 T 的顶点数 $n(T)=1$ 。
- 递归步: 如果 T_1 和 T_2 是满二叉树, 且 $T = T_1 \cdot T_2$, 那么 T 的顶点数 $n(T) = n(T_1) + n(T_2) + 1$ 。

4.3.3 结构归纳法

- 定理2：如果 T 是满二叉树，那么 $n(T) \leq 2^{h(T)+1} - 1$
- 证明：我们用结构归纳法证明这个不等式。
- 基础步：如果满二叉树 T 只含一个根结点 r ，那么 $n(T)=1$ ，
 $h(T)=0$ 。满足 $n(T) = 1 \leq 2^{0+1} - 1 = 1$ 。
- 归纳步：假设 $T = T_1 \cdot T_2$ ， T_1 和 T_2 是满二叉树，满足： $n(T_1) \leq 2^{h(T_1)+1} - 1$ 以及 $n(T_2) \leq 2^{h(T_2)+1} - 1$ ，由 $n(T)$ 和 $h(T)$ 的递归公式：

4.3.3 结构归纳法

- $n(T) = 1 + n(T_1) + n(T_2)$, $h(T) = 1 + \max(h(T_1), h(T_2))$, 有
- $n(T) = 1 + n(T_1) + n(T_2)$
- $\leq 1 + (2^{h(T_1)+1} - 1) + (2^{h(T_2)+1} - 1)$
- $\leq 2 \cdot \max(2^{h(T_1)+1}, 2^{h(T_2)+1}) - 1$
- $= 2 \cdot 2^{\max(h(T_1), h(T_2))+1} - 1$
- $= 2 \cdot 2^{h(T)} - 1$
- $= 2^{h(T)+1} - 1$
- 证毕。

4.4 递归算法

- 定义：一个算法是递归的，是说，它可以通过求解具有更小的输入的同一问题来解决问题。
- 用一系列归约来求解原问题的解，直到把问题归约到解已知的某个初始情形为止
- 例1：给出计算 $n!$ 的递归算法，其中 n 是非负整数
- 例如，两个正整数 a 和 b 的最大公因子问题，归约到 $b \bmod a$ 和 a 的最大公因子问题

4.4 递归算法

- 解：因为 $n! = n \cdot (n - 1)!$ 且 $0! = 1$ ，故设计以下递归算法。
- 算法1：计算 $n!$ 的递归算法。
- PROCEDURE factorial (n: nonnegative integer);
- IF $n=0$ THEN factorial(n) := 1
- ELSE factorial(n) := $n \cdot \text{factorial}(n - 1)$;
- *计算 $4!$ 的步骤： $4! = 4 \cdot 3!$, $3! = 3 \cdot 2!$, $2! = 2 \cdot 1!$, $1! = 1 \cdot 0!$,
- 由 $0! = 1$ ，得 $1! = 1 \cdot 1 = 1$, $2! = 2 \cdot 1 = 2$, $3! = 3 \cdot 2 = 6$, $4! = 4 \cdot 6 = 24$ 。

4.4 递归算法

- 例2：给出计算 a^n 的递归算法，其中 a 是非0实数， n 是非负整数。
- 解：由于 $a^n = a \cdot a^{n-1}$ ，且 $a^0 = 1$ ，得到递归算法。
- 算法2：计算 a^n 的递归算法。
- PROCEDURE power (a : nonzero real number, n : nonnegative integer);
- IF $n=0$ THEN power(a , n) := 1
- ELSE power(a , n) := $a \cdot \text{power}(a, n - 1)$;

4.4 递归算法

- 例3：设计递归算法计算 $b^n \bmod m$ ，其中 b , n 和 m 都是整数，满足： $m \geq 2, n \geq 0$ 和 $1 \leq b < m$ 。
- 解：由递归式 $b^n \bmod m = (b \cdot (b^{n-1} \bmod m)) \bmod m$ ，和 $b^0 \bmod m = 1$ ，我们可以得到递归算法。然而，我们有更有效的计算方法。根据递归式：
 - $b^n \bmod m = (b^{n/2} \bmod m)^2 \bmod m$ ，其中 n 是偶数，
和 $b^n \bmod m = \left((b^{\lfloor n/2 \rfloor} \bmod m)^2 \bmod m \cdot b \bmod m \right) \bmod m$ ，其中 n 是奇数。

4.4 递归算法

- 算法3：指数求模运算的递归算法。
- PROCEDURE mpower (b, n, m: integers with $m \geq 2, n \geq 0$);
- IF $n=0$ THEN
- mpower(b, n, m) := 1
- ELSE IF n is even THEN
- mpower(b, n, m) := mpower(b, $\frac{n}{2}$, m)² mod m
- ELSE
- mpower(b, n, m) := (mpower(b, $\lfloor \frac{n}{2} \rfloor$, m)² mod m ·
b mod m) mod m ;

4.4 递归算法

- *例如：输入 $b=2, n=5, m=3$
- $\text{mpower}(2,5,3)=(\text{mpower}(2,2,3)^2 \bmod 3 \cdot 2 \bmod 3) \bmod 3,$
- $\text{mpower}(2,2,3)=\text{mpower}(2,1,3)^2 \bmod 3$, 再算
- $\text{mpower}(2,1,3)=(\text{mpower}(2,0,3)^2 \bmod 3 \cdot 2 \bmod 3) \bmod 3,$
- $\text{mpower}(2,0,3)=1$.再往回算,
- $\text{mpower}(2,1,3)=(1^2 \bmod 3 \cdot 2 \bmod 3) \bmod 3 = 2$,
故
- $\text{mpower}(2,2,3)= 2^2 \bmod 3 = 1$, 最后,
- $\text{mpower}(2,5,3)= (1^2 \bmod 3 \cdot 2 \bmod 3) \bmod 3 = 2^{68}.$

4.4 递归算法

- 例4：给出递归算法，求两个非负整数 a 和 b 的最大公因子，其中 $a < b$ 。
- 解：由于我们有递归式： $\gcd(a, b) = \gcd(b \bmod a, a)$ 以及 $\gcd(0, b) = b$ 。我们得到递归算法：
- 算法4：求 $\gcd(a, b)$ 的递归算法。
- PROCEDURE $\gcd(a, b$: nonnegative integers with $a < b$);
- IF $a = 0$ THEN $\gcd(a, b) := b$
- ELSE $\gcd(a, b) := \gcd(b \bmod a, a)$;

4.4 递归算法

- 例5：用递归算法表示线性查找算法。
- 解：在序列 $a_1, a_2, \dots, a_n$ 中查找元素 x ，在第 i 步， x 和 a_i 比较，如果 $x = a_i$ ，则返回 x 的位置 i ，如果 x 不等于 a_i ，则在剩下的序列 $a_{i+1}, a_{i+2}, \dots, a_n$ 中查找，我们设立递归过程 $\text{Search}(i, j, x)$ 在 $a_i, a_{i+1}, \dots, a_j$ 中查找 x 。初始输入为： $(1, n, x)$ 。递归算法如下：

4.4 递归算法

- 算法5：递归的线性查找算法。
- PROCEDURE Search (i, j, x : integers, $1 \leq i \leq j \leq n$);
- IF $a_i = x$ THEN
- location := i
- ELSE IF $i = j$ THEN
- location := 0
- ELSE
- Search($i+1, j, x$);

4.4 递归算法

- 例6：构造递归的二分查找算法。
- 解：在递增序列 $a_1, a_2, \dots, a_n$ 中查找元素 x 。
- 算法6：递归的二分查找算法：
- Procedure binary search(i, j, x)
- $m := \lfloor (i + j) / 2 \rfloor$
- If $x = a_m$ then
- return m
- Else if $(x < a_m \text{ and } i < m)$ then
- return binary search($i, m-1, x$)
- Else if $(x > a_m \text{ and } j > m)$ then
- return binary search($m+1, j, x$)
- Else return 0

{输出是 $a_1, a_2, \dots, a_n$ 中 x 的位置，如果有 x ；否则为0}

4.4 递归算法

- PROCEDURE binary search(i, j, x : integers, $1 \leq i \leq j \leq n$);
- BEGIN
- $m := \lfloor (i + j)/2 \rfloor$;
- IF $x = a_m$ THEN
- location := m
- ELSE IF ($x < a_m$ and $i < m$) THEN
- binary search($i, m - 1, x$)
- ELSE IF ($x > a_m$ and $j > m$) THEN
- binary search ($m + 1, j, x$)
- ELSE location := 0;

4.4 递归算法

- 证明算法的正确性
- 例7：证明：计算实数 a 的幂的算法2是正确的。

4.4 递归算法

- 解：我们对指数 n 用数学归纳法证明。
- 当 $n=0$ 时，算法第一步求得 $\text{power}(a, 0)=1$ ，因为 $a^0 = 1$ ($a \neq 0$)，故算法正确。
- 设当 $n=k$ 时，算法计算正确，即 $\text{power}(a, k)=a^k$ 。
- 当 $n=k+1$ 时，算法求得 $\text{power}(a, k+1)=a \cdot \text{power}(a, k)$ ，因为 $a^{k+1} = a \cdot a^k$ ，又由归纳假设， $\text{power}(a, k+1)=a \cdot \text{power}(a, k) = a \cdot a^k = a^{k+1}$ ，故算法正确。

4.4 递归算法

- 例8：证明：指数取模运算的算法5（下一页）正确
- 计算 $b^n \bmod m$ 。将 n 写成二进制，我们算 $b \bmod m$, $b^2 \bmod m, b^4 \bmod m, \dots, b^{2^{k-1}} \bmod m$, 再对所有 $a_j = 1$ 的项，将 $b^{2^j} \bmod m$ 乘在一起，再取 $\bmod m$ 运算

4.4 递归算法

- 算法5: (指数取模运算)
- PROCEDURE modular exponentiation (b : integer;
- $n = (a_{k-1}a_{k-2} \cdots a_1a_0)_2$, m : positive integer);
- $x := 1$;
- $power := b \bmod m$;
- FOR $i := 0$ TO $k - 1$ DO
- BEGIN
- IF $a_i = 1$ THEN $x := (x \cdot power) \bmod m$;
- $power := (power \cdot power) \bmod m$;
- END;
- { x equals $b^n \bmod m$ }

4.4 递归算法

- 解：我们对指数 n 用强归纳法证明。
- 设 b 和 m 是整数且 $m \geq 2$ 。当 $n=0$ 时，算法求得 $\text{mpower}(b,n,m)=1$ ，因为 $b^0 \bmod m = 1$ ，这一步正确。
- 设当 $0 \leq j < k$ 时， $\text{mpower}(b,j,m) = b^j \bmod m$ 。现在设 $n=k$ ，
- 当 k 是偶数时，根据归纳假设，有 $\text{mpower}(b,k,m)=$
$$\text{mpower}\left(b, \frac{k}{2}, m\right)^2 \bmod m = \left(b^{\frac{k}{2}} \bmod m\right)^2 \bmod m = b^k \bmod m.$$
- 当 k 是奇数时，由归纳假设，有 $\text{mpower}(b,k,m)=$
$$\left(\text{mpower}\left(b, \left\lfloor \frac{k}{2} \right\rfloor, m\right)^2 \bmod m \cdot b \bmod m\right) \bmod m =$$

$$\left(\left(b^{\left\lfloor \frac{k}{2} \right\rfloor} \bmod m\right)^2 \bmod m \cdot b \bmod m\right) \bmod m = b^{2\lfloor k/2 \rfloor + 1} \bmod m = b^k \bmod m.$$

故算法正确。

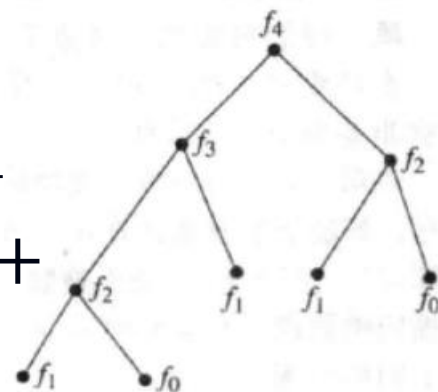
机器人与信息自动化研究所

4.4 递归算法

- 递归和迭代(循环)(recursion and iteration)
- 从函数在一个或多个整数点的函数值开始，再连续地应用递归定义一个一个地求得函数在较大整数点的函数值，这过程称为迭代。

4.4 递归算法

- 算法7：求Fibonacci数的递归算法
- PROCEDURE fibonacci (n: nonnegative integer);
- IF $n = 0$ THEN fibonacci(0) := 0
- ELSE IF $n = 1$ THEN fibonacci(1) := 1
- ELSE fibonacci(n) := fibonacci(n - 1) + fibonacci(n - 2);
- *在递归的计算中，由于 $f_n = f_{n-1} + f_{n-2}$ ，某些 f_i 的计算有重复。例如：计算 f_4 ，其中 f_2 被重复计算(见图).
- *用非递归的循环计算，有时可以消除重复计算。



4.4 递归算法

- 算法8：用循环计算Fibonacci数。
- PROCEDURE iterative fibonacci (n: nonnegative integer);
- IF $n = 0$ THEN $y := 0$
- ELSE BEGIN
- $x := 0$;
- $y := 1$;
- FOR $i := 1$ TO $n - 1$ DO
- BEGIN
- $z := x + y$;
- $x := y$;
- $y := z$;
- END
- END;
- {y is the nth Fibonacci number}.

作业14

- 1. 给出以下序列 $\{a_n\}$ 的递归定义：
- (a) $a_n = 4n - 2$; (b) $a_n = n(n + 1)$; (c) $a_n = 1 + (-1)^n$;
- 2. 用结构归纳法证明： $n(T) \geq 2h(T) + 1$, 其中 T 是一棵满二叉树， $n(T)$ 是 T 的顶点数，并且 $h(T)$ 是 T 的高度。
- 3. 设计一个递归算法计算 n^2 ，其中 n 是非负整数. 然后证明此算法是正确的。
- 4. 设计递归算法计算以下序列中第 k 项(k 是非负整数): $a_0 = 1, a_1 = 2$, 且 $a_n = a_{n-1} \cdot a_{n-2} (n = 2, 3, \dots)$ 。然后设计一个循环算法，计算该序列的第 k 项。

The End



机器人与信息自动化研究所

Institute of Robotics & Automatic Information System



南開大學
Nankai University